GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 07.
JSON

LM
Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es
2025/2026
Versión:260404.1334

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Introducción a JSON **5**
1.1 ¿Qué es JSON? 5
2. Sintaxis Básica de JSON **5**
3. Anidamiento en JSON **7**
4. JSON vs XML **8**
5. Manejo de JSON con JavaScript **8**
6. Los dos sabores de JSON: Objetos y Arrays **10**
6.1 El que empieza por llaves { (El Objeto) 11
6.2 El que empieza por corchetes [(El Array) 12
6.3 El JSON con índices numéricos como claves (Empieza por { "0": ... }) 12
6.4 Viene de un fetch 13
7. Ejemplo Práctico Final **15**
8. Ejercicios y Prácticas **16**
9. Resumen **18**
10. Bibliografía **19**

UD07. JSON

1. INTRODUCCIÓN A JSON

Comprender qué es JSON, por qué se usa, su sintaxis básica y cómo trabajar con él usando JavaScript.

1.1 ¿Qué es JSON?

JSON significa *JavaScript Object Notation* (Notación de Objetos de JavaScript). Aunque lleve "JavaScript" en el nombre, **es solo un formato de texto**.

Imaginad que JSON es como el "inglés" de la programación: es el idioma universal que usan diferentes lenguajes (PHP, Python, Java, JS) para pasarse datos entre ellos sin volverse locos.

¿Por qué no pasamos simplemente objetos de JavaScript?

Porque un servidor escrito en Python no tiene ni idea de qué es un objeto de JS. Sin embargo, ambos saben leer texto plano. JSON es esa traducción a texto.

- **¿Por qué es importante?:**
  - Es **legible** tanto para humanos como para máquinas.
  - Es **compacto** (menos "ruido" que otros formatos como XML).
  - Es **independiente del lenguaje de programación**. Aunque nació de JavaScript, se puede usar con *cualquier* lenguaje (Python, Java, PHP, C#, etc.).
  - Es el formato más **comúnmente usado** hoy en día para APIs web (servicios que permiten que una aplicación hable con otra).

2. SINTAXIS BÁSICA DE JSON

JSON se construye sobre dos estructuras principales:

1. **Objetos:** Representan colecciones de pares **clave-valor** (propiedades). Es como una lista de características de una cosa.
   - Se definen con **llaves {}**.

- Cada par clave-valor se separa por **comas ,**.
- La **clave** (el "nombre" de la propiedad) siempre es una **cadena de texto** (entre comillas dobles ").
- El **valor** puede ser cualquiera de los tipos de datos JSON (ver más abajo).
- La clave y el valor se separan por **dos puntos :**.

```
JSON
{
  "nombre": "Iron Man",
  "identidadSecreta": "Tony Stark",
  "esHumano": true,
  "edad": 45,
  "poderes": null
}
```

2. **Arrays (Listas):** Representan colecciones ordenadas de **valores**. Es como una lista de elementos.
   - Se definen con **corchetes []**.
   - Los **valores** (los elementos de la lista) se separan por **comas ,**.
   - Los valores pueden ser de **diferentes tipos** (incluso otros objetos o arrays).

```
JSON
[
  "Repulsores",
  "Armadura de vuelo",
  "Unibeam",
  "Misiles"
]
```

Tipos de Datos en JSON:

Los valores en JSON pueden ser de los siguientes tipos:

- **String (Cadena):** Texto, siempre entre comillas dobles "".

```
JSON
"Esto es una cadena de texto"
```

- **Number (Número):** Enteros o decimales. Sin comillas.

```
JSON
42
3.1416
```

- **Boolean (Booleano):** Solo true o false. Sin comillas.

```
JSON
true
false
```

- **null:** Representa la ausencia intencionada de cualquier valor. Sin comillas.

```
JSON
null
```

- **Object (Objeto):** Otro objeto JSON anidado.
  JSON
  ```
  { "ciudad": "Nueva York" }
  ```

- **Array (Array):** Otro array JSON anidado.
  JSON
  ```
  [ 1, 2, 3 ]
  ```

Reglas de Sintaxis Clave:

- Todo el contenido es un Objeto {} o un Array [] principal.
- Las claves de los objetos SIEMPRE van entre comillas dobles "".
- Las cadenas de texto (valores) SIEMPRE van entre comillas dobles "".
- Los números, booleanos (true, false) y null NO llevan comillas.
- Los pares clave-valor en objetos se separan por coma ,.
- Los elementos en arrays se separan por coma ,.
- NO puede haber una coma extra al final del último elemento/par (aunque algunos lenguajes/parsers lo perdonan, la especificación estándar no).
- Se pueden usar espacios en blanco (espacios, saltos de línea, tabulaciones) para formatear, pero no afectan el significado.

3. ANIDAMIENTO EN JSON

La verdadera potencia de JSON viene de poder combinar estas estructuras. Puedes tener objetos dentro de arrays, arrays dentro de objetos, objetos dentro de objetos, etc.

Ejemplo: Un array de objetos (una lista de personas)

JSON

```
[
  {
    "id": 1,
    "nombre": "Peter Parker",
    "alias": "Spider-Man",
    "ciudad": "Nueva York",
    "poderes": [
      "Telarañas",
      "Sentido arácnido",
      "Súper fuerza"
    ],
    "equipo": null
  },
  {
```

```
    "id": 2,
    "nombre": "Steve Rogers",
    "alias": "Capitán América",
    "ciudad": "Brooklyn",
    "poderes": [
      "Súper fuerza",
      "Agilidad",
      "Escudo indestructible"
    ],
    "equipo": "Los Vengadores"
  }
]
```

- Aquí tenemos un Array principal [].
- Cada elemento del array es un Objeto {} que representa a un superhéroe.
- Dentro de cada objeto, hay propiedades con diferentes tipos de valores: Number (id), String (nombre, alias, ciudad, equipo), Array (poderes), y null (equipo en el primer objeto).
- El array poderes contiene Strings.

4. JSON VS XML

En la siguiente unidad veremos XML, pero aquí va una pequeña justificación de por que ha tenido tanto éxito JSON.

- XML fue popular antes que JSON para el intercambio de datos.
- XML usa etiquetas (<nombre>...</nombre>). Es más "hablador" (verbose).
- JSON es más compacto y generalmente más fácil de parsear (procesar) con código.
- Hoy en día, JSON es mucho más común para APIs web.

5. MANEJO DE JSON CON JAVASCRIPT

JavaScript tiene soporte nativo para trabajar con JSON. ¡Es muy fácil!
¿Por qué JavaScript?: Porque JSON nació de JavaScript, y en el desarrollo web es el lenguaje principal en el navegador para interactuar con APIs que a menudo devuelven datos en formato JSON.
Conceptos clave:

- **JSON String:** El texto plano que recibimos o enviamos (cumple la sintaxis JSON).

```
JavaScript
// Esto es una cadena de texto en JS, la comilla grave ` es para saltos
//de línea ;)
let jsonString = `
{
  "producto": "Laptop",
  "precio": 1200,
  "enStock": true
}`
```

- **JavaScript Object/Array:** La estructura de datos real con la que trabajamos en nuestro código JS.
  ```
  JavaScript
  // Esto es un objeto de JavaScript
  let jsObject = {
    producto: "Laptop",
    precio: 1200,
    enStock: true
  }
  ```
  Fíjate que en el **objeto JS**, las claves (si son nombres de variables válidos) NO necesitan comillas, a diferencia de **JSON**.

Tres métodos mágicos:

La clase global JSON en JavaScript proporciona estos métodos:

1. **JSON.parse(jsonString):**
   - **¿Qué hace?:** Convierte una **cadena de texto JSON** a un **objeto o array de JavaScript**.
   - **¿Cuándo se usa?:** Cuando **ya tienes el texto JSON guardado en una variable** en tu ordenador y quieres trabajar con ellos en tu código JS como estructuras de datos normales.

     ```
     JavaScript
     let jsonString = '{"nombre": "Braulio", "edad": 30}';
     let jsObject = JSON.parse(jsonString);

     console.log(jsObject.nombre); // Salida: Braulio
     console.log(jsObject.edad);   // Salida: 30

     // Ejemplo con Array
     let jsonArrayString = '[{"item": "manzana"}, {"item": "banana"}]';
     let jsArray = JSON.parse(jsonArrayString);

     console.log(jsArray[0].item); // Salida: manzana
     ```

   - **Importante:** Si el jsonString no es un JSON válido, JSON.parse() lanzará un error. Conviene usar try...catch en código real.
2. **JSON.stringify(jsObjectOrArray)**:

- **¿Qué hace?:** Convierte un **objeto o array de JavaScript** a una **cadena de texto JSON**.
- **¿Cuándo se usa?:** Cuando quieres enviar datos (por ejemplo, a una API para guardar algo) o guardar datos en un formato estándar de texto.

```
JavaScript
let jsObject = {
  nombre: "Ana",
  profesion: "Desarrolladora",
  hobbies: ["leer", "codificar"]
};

let jsonString = JSON.stringify(jsObject);

console.log(jsonString);
// Salida: {"nombre":"Ana","profesion":"Desarrolladora","hobbies":
["leer","codificar"]}
// (Nota: La salida es una única línea sin espacios, compacta para la
transferencia)
```

- **Nota:** JSON.stringify automáticamente pone las comillas dobles donde corresponden en el JSON string resultante.

3. **response.JSON()**:
   - **¿Qué hace?:** Este método no es de JavaScript puro, sino que pertenece a la **API Fetch** (la herramienta del navegador para hacer peticiones a internet).Lee el "cuerpo" (body) de una respuesta de red que viene de internet y, cuando termina de descargarlo, lo convierte en un objeto JS.
   - **¿Cuándo se usa?: Exclusivamente** después de hacer un `fetch()`

```
fetch('https://api.ejemplo.com/datos')
  .then(respuesta => {
    // Los datos están llegando de internet...
    // respuesta.json() ESPERA a que lleguen todos y luego los convierte
    return respuesta.json();
  })
  .then(datos => console.log(datos));
```

6. LOS DOS SABORES DE JSON: OBJETOS Y ARRAYS

¿Cómo recorro un JSON recibido por una API? Depende de cómo empiece el JSON ({, [, o si tiene números).

Característica	.forEach() productos.forEach((producto) =>	for...in for (let clave in perfil) {
¿Para qué se usa?	Exclusivo para **Arrays []** (Listas).	Diseñado para **Objetos {}** (Diccionarios).
¿Qué extrae?	Extrae directamente el **Valor** (el elemento en sí).	Extrae la **Clave** (el nombre de la propiedad).
Analogía	Sacar uno a uno los objetos de una caja.	Leer las etiquetas de las carpetas de un archivador.
¡Peligro!	Si intentas usarlo en un Objeto {}, **dará error** (is not a function).	Si lo usas en un Array, te devolverá los índices (0, 1, 2...) como texto, lo cual es ineficiente y propenso a errores.

6.1 El que empieza por llaves { (El Objeto)

Representa un **único elemento** o entidad con sus características. NO es un array, es un **objeto**

JSON

```
// 1. Recibimos los datos de la red como un texto plano (String)
// Fíjate que todo está envuelto en comillas invertidas (`).
    const datosRecibidos = `
    {
          "nombreEquipo": "Los Vengadores",
          "base": "Torre Stark",
          "estadoAlerta": true,
          "miembros": [
              { "alias": "Iron Man", "nombreReal": "Tony Stark", "poder": 95 },
              { "alias": "Thor", "nombreReal": "Thor Odinson", "poder": 100 },
              { "alias": "Viuda Negra", "nombreReal": "Natasha Romanoff",
      "poder": 80 }
          ]
    }
    `;


Javascript
      // 2. Transformamos el texto en un Objeto Real de JavaScript
    const equipo = JSON.parse(datosRecibidos);


    // 3. Recorremos el Objeto para ver sus propiedades y valores
    for (let clave in equipo) {
        console.log(`Propiedad: ${clave} -> Valor:`, equipo[clave]);
    }
```

6.2 El que empieza por corchetes [(El Array)

Representa una **lista de elementos**. Casi siempre es una lista que contiene objetos dentro. Es lo más común cuando pedimos un listado a una API.

```
// 1. Recibimos los datos como un String (texto).
// ¡OJO AL PRIMER CARÁCTER! Es un [, por lo tanto es una lista de elementos.
const inventarioJSON = `
[
  { "id": 1, "nombre": "Espada de Hierro", "tipo": "Arma", "poder": 45 },
  { "id": 2, "nombre": "Poción Menor", "tipo": "Consumible", "curacion": 50 },
  { "id": 3, "nombre": "Escudo de Madera", "tipo": "Defensa", "poder": 15 }
]
`;

// 2. Transformamos el texto en un Array Real de JavaScript
const mochila = JSON.parse(inventarioJSON);

console.log("Abriendo el inventario...");
console.log(`Tienes ${mochila.length} objetos en la mochila.`);
console.log("-----------------------------------");

// 3. Como la variable "mochila" YA ES un Array (porque el JSON empezaba por [ ),
// usamos .forEach() DIRECTAMENTE sobre ella.
mochila.forEach(item => {

    // Imprimimos el nombre de cada objeto
    console.log(`${item.nombre} (Tipo: ${item.tipo})`);

    // Añadimos un poco de lógica dependiendo del tipo de objeto
    if (item.tipo === "Arma" || item.tipo === "Defensa") {
        console.log(`   Poder de equipo: ${item.poder} pts.`);
    } else if (item.tipo === "Consumible") {
        console.log(`   Restaura: ${item.curacion} puntos de vida.`);
    }
});
```

6.3 El JSON con índices numéricos como claves (Empieza por { "0": ... })

A veces os encontraréis con APIs (o bases de datos como Firebase) que devuelven algo muy raro: un objeto que parece un array, pero es un objeto cuyas claves son números en formato texto.

```
JavaScript

const alumnosClase = {
  "0": { "nombre": "Laura", "nota": 8.5 },
```

```
  "1": { "nombre": "Carlos", "nota": 6.0 },
  "2": { "nombre": "Marta", "nota": 9.2 }
};
```

Solución: Como empieza por {, sigue siendo un objeto. La técnica más moderna (y profesional) para convertir esto en algo manejable es usar **Object.values()**, que extrae solo los valores y los mete en un Array de verdad, permitiéndonos usar .forEach().

JavaScript

```
// Convertimos el objeto raro en un Array normal
const arrayAlumnos = Object.values(alumnosClase);

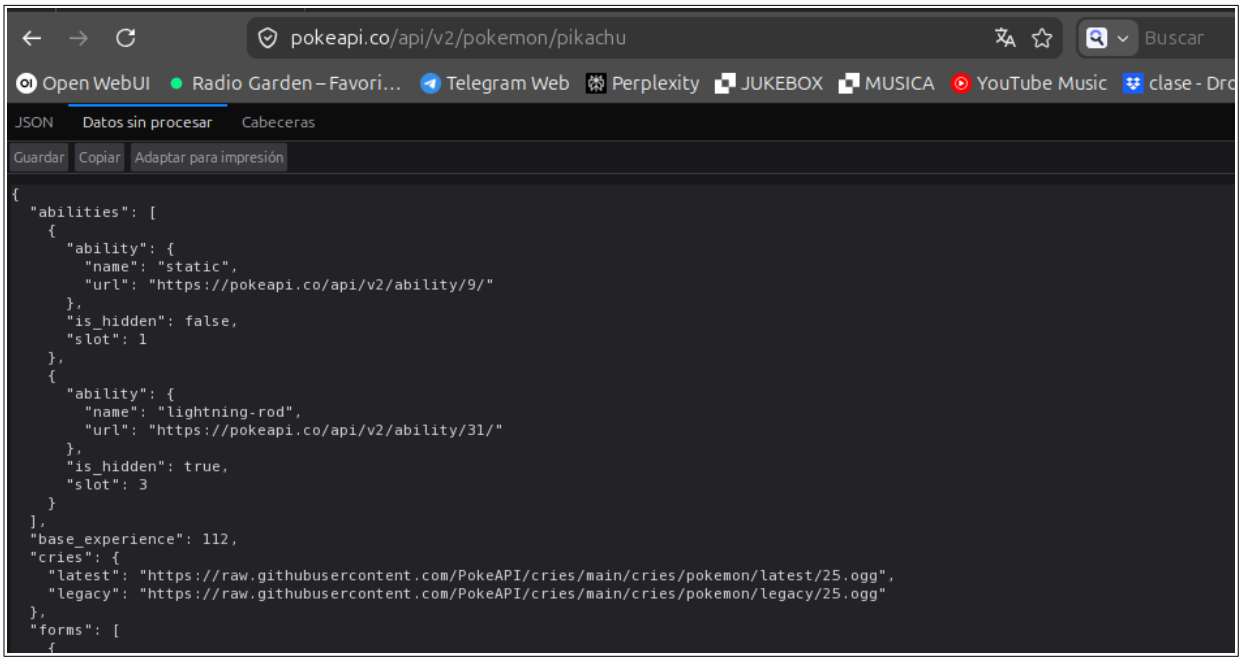
// Ahora ya podemos recorrerlo fácilmente
arrayAlumnos.forEach(alumno => {
  console.log(`${alumno.nombre} ha sacado un ${alumno.nota}`);
});
```

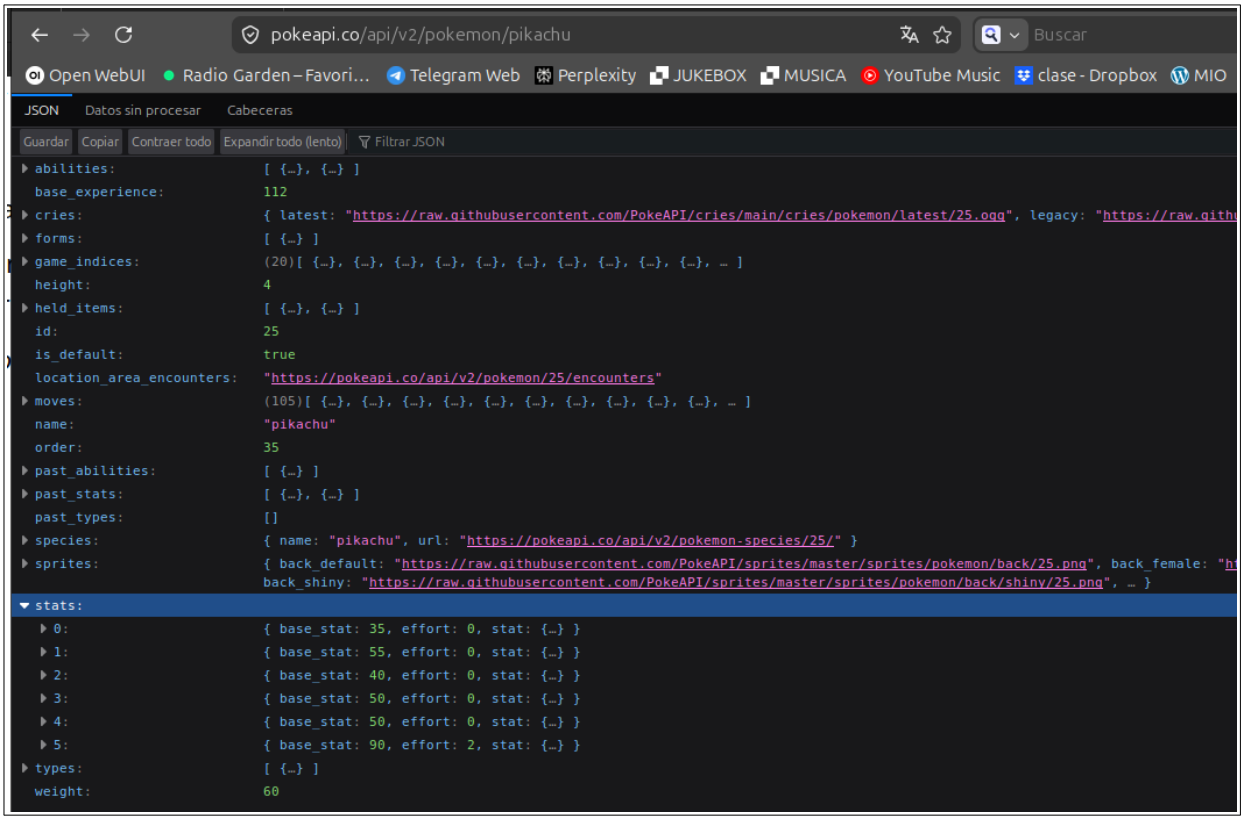
6.4 Viene de un fetch

```
  // Pedimos los datos específicos de Pikachu
     fetch('https://pokeapi.co/api/v2/pokemon/pikachu')
     .then(respuesta => respuesta.json())
     .then(pikachu => {
         // Fíjate en la consola: el primer carácter es una {
         console.log("JSON recibido (Objeto):", pikachu);

     // Aquí NO podemos usar forEach.
     // Como es un objeto, accedemos directamente a sus propiedades con el punto (.)
         console.log(`⚡ Nombre: ${pikachu.name}`);
         console.log(`⚖️ Peso: ${pikachu.weight / 10} kg`);

         // Si quisiéramos recorrer sus estadísticas, buscaríamos el Array que hay
DENTRO del objeto:
         console.log("--- Estadísticas ---");
         pikachu.stats.forEach(estadistica => {
             //console.log(`${estadistica.stat.name}: ${estadistica.base_stat}`);
         });
     });
```


pokeapi.co/api/v2/pokemon/pikachu
Buscar
Open WebUI
Radio Garden – Favori...
Telegram Web
Perplexity
JUKEBOX
MUSICA
YouTube Music
JSON
Datos sin procesar
Cabeceras
Guardar
Copiar
Adaptar para impresión
{
"abilities": [
{
"ability": {
"name": "static",
"url": "https://pokeapi.co/api/v2/ability/9/"
},
"is_hidden": false,
"slot": 1
},
{
"ability": {
"name": "lightning-rod",
"url": "https://pokeapi.co/api/v2/ability/31/"
},
"is_hidden": true,
"slot": 3
}
],
"base_experience": 112,
"cries": {
"latest": "https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/latest/25.ogg",
"legacy": "https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/legacy/25.ogg"
},
"forms": [


pokeapi.co/api/v2/pokemon/pikachu
Buscar
Open WebUI
Radio Garden – Favori...
Telegram Web
Perplexity
JUKEBOX
MUSICA
YouTube Music
clase - Dropbox
MIO
JSON
Datos sin procesar
Cabeceras
Guardar
Copiar
Contraer todo
Expandir todo (lento)
Filtrar JSON
abilities: [{…}, {…}]
base_experience: 112
cries: { latest: "https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/latest/25.ogg", legacy: "https://raw.gith
forms: [{…}]
game_indices: (20)[{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, …]
height: 4
held_items: [{…}, {…}]
id: 25
is_default: true
location_area_encounters: "https://pokeapi.co/api/v2/pokemon/25/encounters"
moves: (105)[{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, …]
name: "pikachu"
order: 35
past_abilities: [{…}]
past_stats: [{…}, {…}]
past_types: []
species: { name: "pikachu", url: "https://pokeapi.co/api/v2/pokemon-species/25/" }
sprites: { back_default: "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/25.png", back_female: "h
back_shiny: "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/shiny/25.png", … }
stats:
0: { base_stat: 35, effort: 0, stat: {…} }
1: { base_stat: 55, effort: 0, stat: {…} }
2: { base_stat: 40, effort: 0, stat: {…} }
3: { base_stat: 50, effort: 0, stat: {…} }
4: { base_stat: 50, effort: 0, stat: {…} }
5: { base_stat: 90, effort: 2, stat: {…} }
types: [{…}]
weight: 60

7. EJEMPLO PRÁCTICO FINAL

Vamos a juntar todo. Haremos una petición con fetch() a una API pública que nos devuelve un Array de objetos [{}, {}] con datos de usuarios, y los pintaremos en nuestra web.

HTML:

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Lista de Usuarios</title>
    <style>
        body { font-family: sans-serif; background: #f4f4f9; padding: 20px; }
        .grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(200px, 1fr)); gap: 15px; }
        .card { background: white; padding: 15px; border-radius: 8px; box-shadow:
0 4px 6px rgba(0,0,0,0.1); }
        .card h3 { margin: 0 0 10px 0; color: #333; }
        .card p { margin: 5px 0; color: #666; font-size: 14px; }
    </style>
</head>
<body>
    <h1>Directorio de Empleados</h1>
    <div id="contenedor-usuarios" class="grid"></div>

    <script src="app.js"></script>
</body>
</html>
```

JavaScript (app.js):

JavaScript

```
const contenedor = document.getElementById('contenedor-usuarios');

// 1. Hacemos la petición a la API
fetch('https://jsonplaceholder.typicode.com/users')
  .then(respuesta => {
    // 2. Parseamos la respuesta a JSON
    return respuesta.json();
  })
  .then(usuarios => {
    // 3. El JSON es un Array, así que lo recorremos
    console.log("Datos recibidos:", usuarios); // ¡Míralo siempre en consola
primero!

    usuarios.forEach(usuario => {
        const htmlUsuarios =
```

```
        `<div class="card">
            <h3>${usuario.name}</h3>
            <p>${usuario.email}</p>
            <p>${usuario.address.city}</p>
        </div>`
        // 4. Inyectamos en el DOM
        contenedor.innerHTML += htmlUsuarios;
    });
})
.catch(error => console.error("Houston, tenemos un problema:", error));
```

8. EJERCICIOS Y PRÁCTICAS

Ejercicio 1: Creando JSON básico

- Crea un string en formato JSON que represente un objeto {} con tus datos (nombre, edad, ciudad, si eres estudiante).
- Crea un string en formato JSON que represente un array [] con tus 3 asignaturas favoritas.
- Crea un string JSON que represente un objeto {} de un libro, incluyendo título, autor, año de publicación y un array de géneros.

Ejercicio 2: Validando JSON

- Busca un "JSON validator online" (validador JSON en línea).
- Pega tus JSON strings del Ejercicio 1 en el validador para comprobar si son sintácticamente correctos. Corrige cualquier error.
- Intenta introducir errores a propósito (ej: olvidar comillas en una clave, poner coma al final del último elemento) y observa qué dice el validador.

Ejercicio 3: JSON en JavaScript (Parse y Stringify)

- En un archivo .js (o la consola del navegador):
  - Define un string que contenga un JSON de un objeto que represente un coche (marca, modelo, año).
  - Usa JSON.parse() para convertir ese string a un objeto JavaScript.
  - Accede y muestra por consola la marca y el modelo del coche desde el objeto JavaScript.
  - Define un objeto JavaScript que represente a una persona (nombre, país, array de idiomas que habla).
  - Usa JSON.stringify() para convertir ese objeto JavaScript a un string JSON.
  - Muestra por consola el string JSON resultante.

Ejercicio 4: Accediendo a datos anidados

- Dado el siguiente string JSON:
  JSON

```
{
  "empresa": "Tech Solutions",
  "departamentos": [
    {
      "nombre": "Desarrollo",
      "empleados": [
        { "nombre": "Luis", "rol": "Programador Senior" },
        { "nombre": "María", "rol": "Diseñadora UX/UI" }
      ]
    },
    {
      "nombre": "Marketing",
      "empleados": [
        { "nombre": "Elena", "rol": "Especialista SEO" }
      ]
    }
  ]
}
```

- Usa JSON.parse() para convertirlo a un objeto JS.
- Desde el objeto JS, accede y muestra por consola:
  - El nombre de la empresa.
  - El nombre del primer departamento.
  - El nombre del primer empleado del primer departamento.
  - El rol de María.
  - El nombre del segundo departamento.

Ejercicio 5: Modificando y volviendo a Stringify

- Usando el objeto JavaScript del Ejercicio 4:
  - Añade un nuevo empleado al departamento de "Marketing".
  - Cambia el rol de Luis a "Arquitecto de Software".
  - Usa JSON.stringify() para convertir el objeto modificado de nuevo a un string JSON.
  - Muestra el nuevo string JSON por consola (opcionalmente, con formato legible).

Prácticas Opcionales (para profundizar):

- **Práctica 1: Trabajando con una API real (más compleja)**

    - Busca otra API pública y gratuita que devuelva JSON (ej: API de Star Wars SWAPI, API del tiempo, etc.).
    - Usa Workspace para obtener datos de esa API.
    - Explora la estructura del JSON que recibes.
    - Muestra por consola información específica de los datos obtenidos (ej: nombres de planetas, temperatura actual, etc.). Maneja los errores con catch.
- **Práctica 2: Enviando datos JSON (Workspace con método POST)**
    - Busca una API de prueba que permita enviar datos (ej: JSONPlaceholder tiene endpoints para crear posts - /posts).
    - Crea un objeto JavaScript con los datos que quieres enviar (ej: un nuevo post con title y body).
    - Usa JSON.stringify() para convertir el objeto a un string JSON.
    - Usa Workspace con el método POST y configura el body de la petición con el string JSON y la cabecera Content-Type: application/json.
    - Procesa la respuesta (la API de prueba a menudo devuelve el objeto que "creaste" con un ID).

9. RESUMEN

- JSON es un formato estándar para intercambiar datos de forma legible y ligera.
- Se basa en Objetos (pares clave-valor con {}) y Arrays (listas con []).
- Los datos pueden ser String, Number, Boolean, null, Object o Array.
- Las claves y los strings SIEMPRE llevan comillas dobles "".
- JavaScript tiene JSON.parse() para convertir JSON string a objeto/array JS y JSON.stringify() para convertir objeto/array JS a JSON string.
- Workspace es una herramienta común para obtener datos JSON de APIs web.

10. BIBLIOGRAFÍA

- **Validadores/Formateadores online:**
  - JSONLint (jsonlint.com)
  - CodeBeautify JSON Validator (codebeautify.org/jsonvalidator)
- **Documentación MDN (Mozilla Developer Network):**
  - JSON: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON
  - fetch: https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- **APIs públicas de prueba:**
  - JSONPlaceholder (jsonplaceholder.typicode.com) - Ideal para probar GET y POST.
  - SWAPI (swapi.dev) - API de Star Wars (solo GET).